LMDrive: Closed-Loop End-to-End Driving with Large Language Models

2024 CVPR

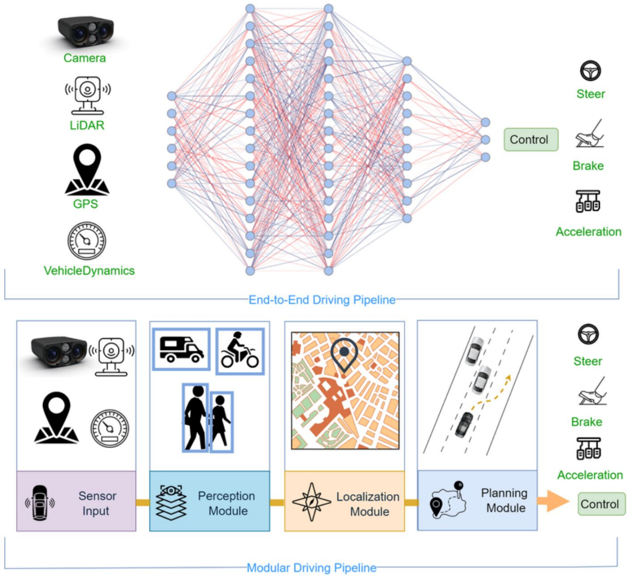
Introduction

LMDrive: Can we build cognitive autonomous driving systems on top of LLMs,
that can interact with human passengers or navigation software simply by natural language?

State-of-the-art autonomous driving systems

End-to-end

Modular system



Solely rely on fixed-format inputs
(sensor data, target waypoints, action commands)

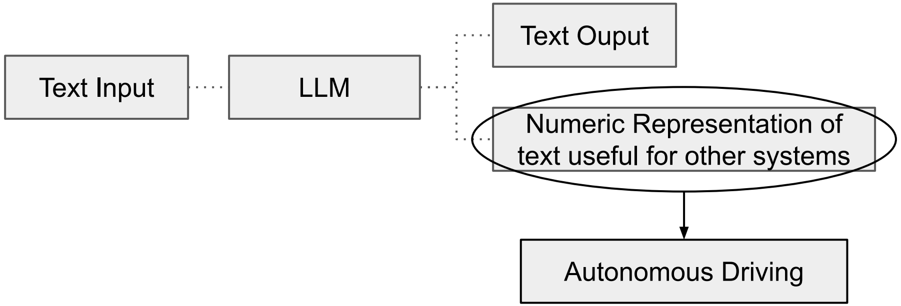
=> Restricts agent's ability to <u>comprehend multi-modal information</u> and to <u>interact with humans and the environments</u>

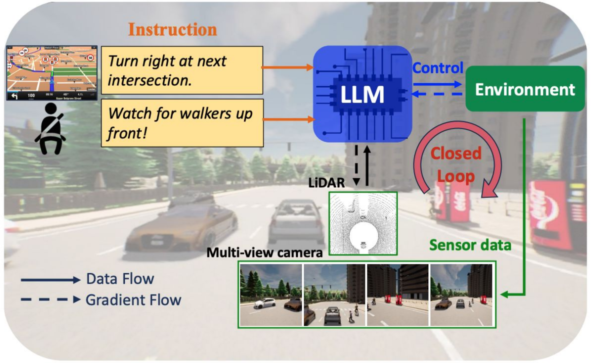
출처 : https://www.researchgate.net/figure/Comparison-between-End-to-End-and-modular-pipelines-End-to-End-is-a-single-pipeline-that_fig3_372248430

Introduction

LMDrive: Can we build cognitive autonomous driving systems on top of LLMs,
that can interact with human passengers or navigation software simply by natural language?

Large language models(LLMs)





Large language models(LLMs) have shown an impressive range of capabilities that approach “Artificial General Intelligence.”
Such capabilities could greatly enhance the safety, controllability, and explainability of autonomous agents.

=> Advanced reasoning in complex situations and efficient interaction with humans

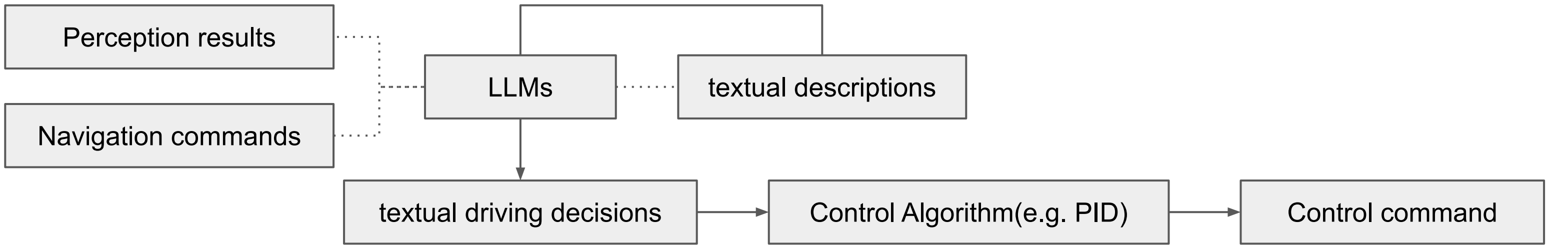
e.g. 1) In long-tail unforeseen events and challenging urban situations (e.g.complex and dense intersections), the AVs can <u>easily survive by following navigation instructions from passengers or navigation software</u>.

2) AVs can adapt to passengers’ sudden notice(e.g. small objects that are easily missed by perception systems) <u>simply via natural language</u>, which was previously non-trial and required a large amount of hand-crafted rules

Contribution

LMDrive: Interacts with the dynamic environment via multi-modal multi-view sensor data and natural language instructions.

Common approach



1) Use LLMs to transform the scene perception results and navigation commands into textual descriptions;
2) Feed these textural descriptions into LLMs to generate textual driving decisions
3) Transfer textual driving decisions into executable control commands.

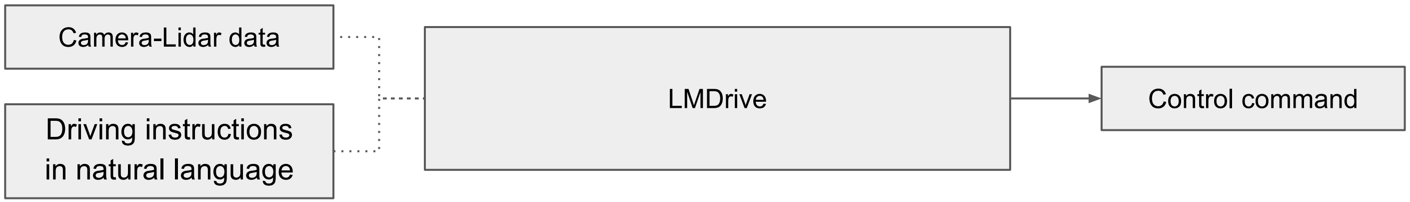
1. Different LLMs tackle sub-tasks individually
   ⇒ Hard to be trained, Low capability to scale with a large amount of data, Not robust to perception errors and uncertainties
2. LLMs in the latter two stages lack access to sensor data
   ⇒ Inaccuracies or missed detections in the first stage can result in significant cumulative errors in the subsequent stages.
3. All other methods to address these issues undergo training and evaluation in the open-loop setting
   ⇒ Actions are generated and evaluated against the expert actions, but not executed in the actual environments.

* For navigation instructions such as “turn right”, the AV agent not only generate a sequence of actions, but also consider the changes brought to the environment.

Contribution

LMDrive: Interacts with the dynamic environment via multi-modal multi-view sensor data and natural language instructions.

Language-guided, end-to-end, close-loop approach



1) Comprehend driving instructions <u>in natural language</u>, and <u>directly generate vehicle control signals</u>
2) A <u>pre-trained LLM model</u> is adopted and <u>kept frozen</u> to maintain its reasoning capability.
3) To adapt the LLM for autonomous driving, <u>multiple camera-LiDAR data encoders</u> and <u>learnable input/output adapters</u> are integrated.

1. LMDrive, end-to-end, closed-loop, language-based autonomous driving framework.
   ⇒ Interacts with the dynamic environment via multi-modal multi-view sensor data and natural language instruction
2. Provide a dataset with about 64K data clips,
   ⇒ Each clip includes one navigation instruction, several notice instructions, a sequence of multi-modal/view sensor data, and control signals.
   The duration of the clip spans from 2 to 20 seconds.
3. Benchmark LangAuto for evaluating the autonomous agents that take language instructions as navigation inputs
   ⇒ Includes misleading/long instructions and challenging driving scenarios.
4. Extensive closed-loop experiments to demonstrate the effectiveness of the proposed framework, and analyze LMDrive

Related work

LMDrive: In a driving system, the vehicle must interact with the environment and respond in real-time, making it essential to consider closed-loop mechanisms.

End-to-end Autonomous Driving

: Imitation training methods(e.g. *UniAD, ThinkTwice, ReasonNet, InterFuser, TCP, LAV*)
: Reinforcement learning methods(e.g. *Latent DRL, Roach, ASAPRL, TaEcRL*)

⇒ Lack the ability to verbally or textually interact with humans(passengers), and usually have low explanatility in the decision-making process.

LLMs in Driving Task

* VLLM - vision encoders for LLMs to interpret images and data in other modalities also

: Vision large language models(VLLM) methods(e.g. *DRIVEGPT4*)

⇒ Framework lacks an input command, the predicted control can not follow the specific navigation command, which denotes that the framework is hard to deploy in real scenarios.

* Directly interpreting and reasoning comprehensive driving situations.

: Transforming the driving situations into textual descriptions as the input for the LLM (e.g. GPT-Driver, languageMPC, LLM-Driver)

⇒ Only considered the driving problem in the open-loop settings, and ignored questions such as cumulative error, temporal action consistency, and end-to-end trainability, which are critical for bringing the models into actual closed-loop driving tasks.

Dataset generation

LMDrive: Intelligent driving agent generating driving actions based on three sources of input

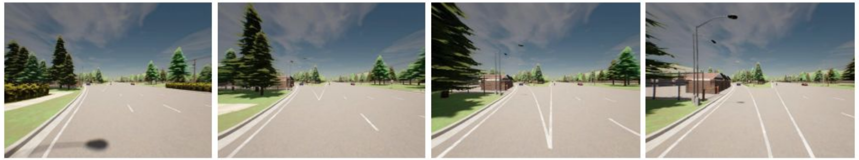
Input sources			
Input sources	Sensor data (Multi-view camera and LiDAR)	Navigation Instructions (e.g. Lane changing, turning)	Human notice instructions (e.g. attention to adversarial events)

Data Collection

Sensor and control data collection; collect data from rule-based expert agent(3M driving frames) with CARLA
: 2.5k routes, 8 towns, and 21 kinds of environmental conditions(e.g.weather, time of the day).
: Four RGB cameras (left, front, right, rear) and one LiDAR.
: The side cameras are angled at 60°, center-cropped front image as an additional focus-view image to capture the status of the distant traffic light.

Parsing and language annotation; Parsing collected data into clips and label with navigation instructions
: Takes a sequence of frames as input, and segments these frames into clips, where each clip corresponds to one navigation instruction.

* Each clip includes sensor data, control signals, the corresponding navigation instruction, and optional notice instructions.



Navigation instruction: Just move to the left and get ready to leave the highway.

Navigation instruction: Turn left at the next T-junction
Notice instruction: Please watch out for the pedestrians up ahead.

Dataset generation

LMDrive: Intelligent driving agent generating driving actions based on three sources of input

Instruction design: Consider three types of navigation instructions(follow, turn, and others) along with one type of notice instruction (consisting of a total of 56 different instructions.)

Diversifying the instructions
- ChatGPT API to generate eight different variants for each type of instruction.
- Each variant carries the same semantic meaning but varies in phrasing.

⇒ More comprehensive coverage and flexibility in language interpretation

Incorporation misleading instructions
- Simulate misleading instructions that violate traffic rules or raise safety concerns and add them to our dataset.

⇒ Improve the robustness of our model against misleading instructions

Connecting multiple instructions
- Construct some consecutive complex instruction data such as "Turn right at this intersection, then go straight to the next intersection and turn right again"

⇒ Simulate real navigation-based driving scenario

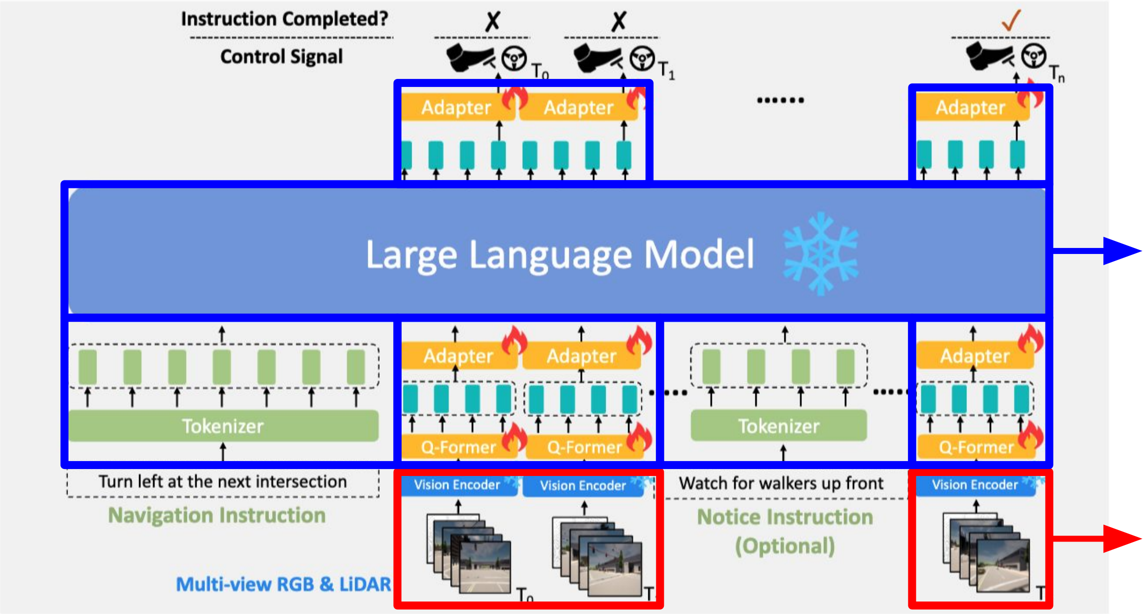
Type	Three randomly chosen instructions of each instruction type
Follow	Maintain your current course until the upcoming intersection. In [x] meters, switch to left lane. Ease on to the left and get set to join the highway.
Turn	After [x] meters, take a left. At the next intersection, just keep heading straight, no turn. You'll be turning left at the next T-junction, alright?
Others	Feel free to start driving. Slow down now. Head to the point, next one's [x] meters ahead, [y] meters left/right.
Notice	Watch for walkers up front. Just a heads up, there's a bike ahead. Please be aware of the red traffic signal directly in front of you.

* Examples of considered navigation instructions

Methodology

LMDrive: A framework that can understand and follow high-level driving instructions through natural language.

The structure of the proposed LMDrive model, which consists of two major components:



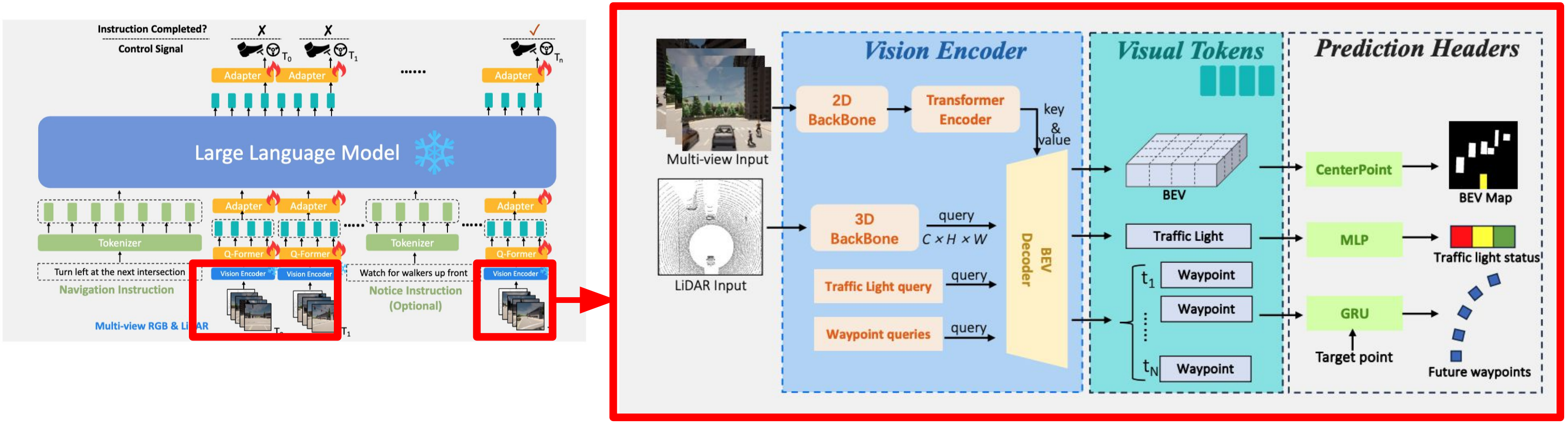
A <u>large language model and its associated component(tokenizer, Q-Former, and adapters)</u> that processes all the historic visual tokens and the language instructions(navigation instruction and optional notice instruction), to predict the control signal and whether the given instruction is completed

A vision encoder <u>processes multi-view multi-modal sensor data(camera and LiDAR)</u> for scene understanding and generating visual tokens

Methodology

LMDrive: A framework that can understand and follow high-level driving instructions through natural language.

Vision Encoder



: Pre-trained *CLIP(Contrastive Language-Image Pretraining)* models are commonly used to align vision and language by encoding image features. However, these models have high computational demands, making them challenging to deploy in real-time AV systems.

: AV perception systems are usually in 3D to include LiDAR input. Inspired by *Interfuser* or *TF++*, multi-view multi-modality vision encoder designed to encode/fuse the sensor data.

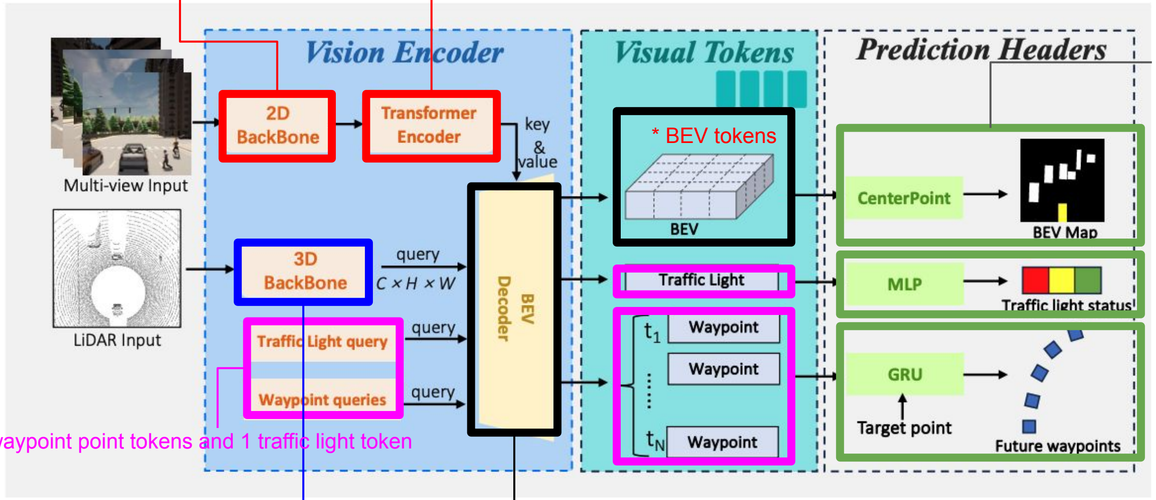
Methodology

LMDrive: A framework that can understand and follow high-level driving instructions through natural language.

Vision Encoder: Pre-trained on perception tasks by adding additional prediction heads, then the encoder is frozen for later use by the large language model.

* ResNet: Extract the feature map

* Transformer: Encode tokens with K_enc layered Multi-Headed Self-Attention



* N waypoint point tokens and 1 traffic light token

* Prediction headers are <u>only used in the pre-training of the vision encoder</u>, and will be discarded in the training of the LLMs and inference of the whole model.

Object detection
: One-stage CenterPoint to predict the bounding boxes and velocity of the objects
: Loss function from Interfuser

Traffic light status detection
: MLP with cross-entropy traffic light state loss

Waypoint prediction
: GRU with l1 loss

* Transformer: Decode tokens with Multi-Headed Self-Attention

* PointPillars: Process the raw point cloud data into ego-centered LiDAR features
* PointNet: Aggregate features and downsample(C×H×W) the feature map to BEV queries

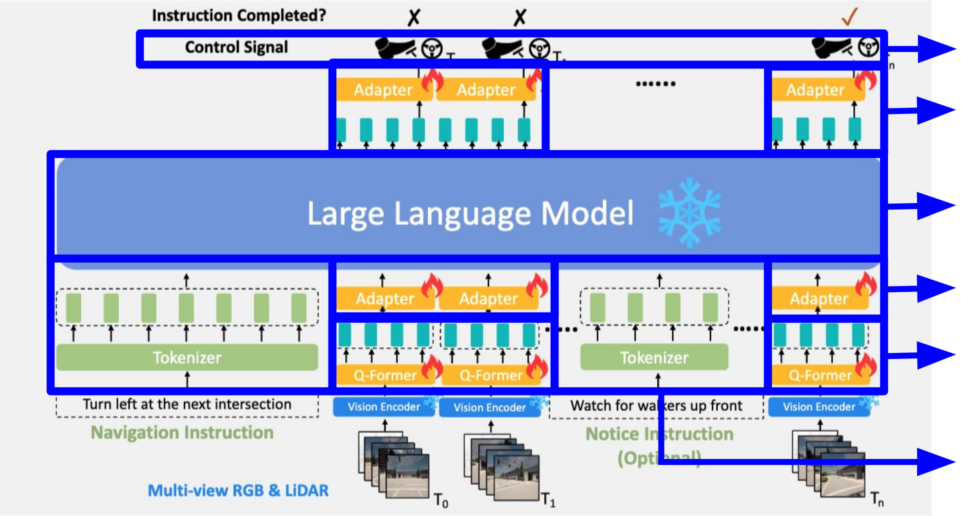
Methodology

LMDrive: A framework that can understand and follow high-level driving instructions through natural language.

LLM for instruction-following auto driving

: Three associated components to bridge LLM with instruction, visual information input, and action prediction
1) a tokenizer, 2) a Q-Former, 3) two adapters.

: <u>Prediction for every historic frame during training</u> to enhance the supervision signal and only the prediction of the latest frame will be executed at inference time. ⇒ Closed-loop



Two PID	; for latitudinal and longitudinal control ; track the heading and velocity of predicted waypoints
2-Layer MLP	; predict future waypoints ; indicate the given instruction has been completed
LLaMA	; predicts the action tokens from input tokens
2-Layer MLP	; convert the tokens to share the same dimension
BLIP-2 (Q-Former)	; reduce the number of visual tokens
LLaMA tokenizer	; convert the instruction in to textual tokens

Methodology

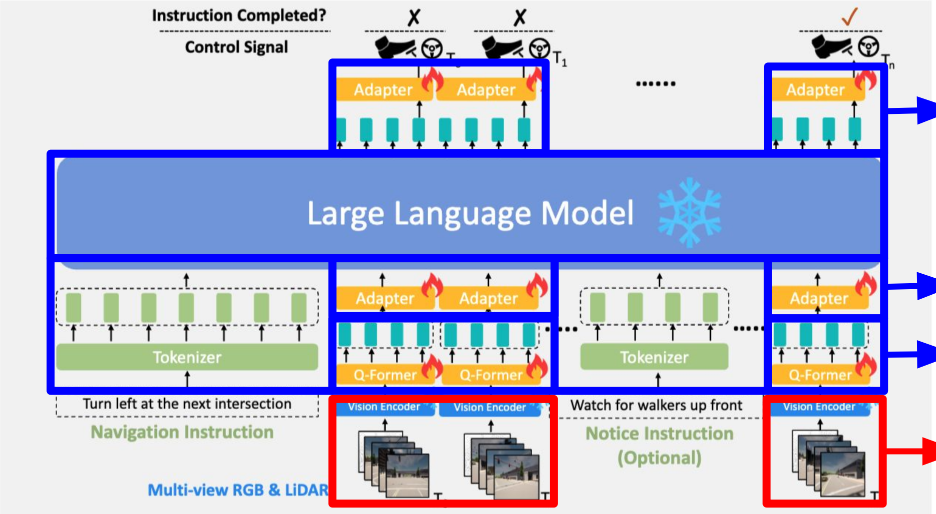
LMDrive: A framework that can understand and follow high-level driving instructions through natural language.

LLM for instruction-following auto driving

: Training details

1) The vision encoder pre-training stage

2) The instruction fine tuning stage(align the instruction/vision and control signal)



l1 waypoint loss; evaluate the performance in predicting future waypoints

②

- Sequence of frames with fixed length(T_max) as input
- Label as ‘completed’ after the misleading instruction is given for 1 second
- Components except Adapter and Q-Former are frozen
- Cross-entropy loss; determines if the current frame finishes the given instruction

①

- Since the instruction annotation process will drop some frames, the raw dataset used(around 3M frames)
- FROZEN after trained

LangAuto Benchmark

LangAuto: First benchmark evaluates closed-loop driving performance under language instructions.

: Previous benchmark(e.g. TOWN05 or longest6) - Navigate the agent with discrete driving commands or target waypoints
: LangAuto - Only provides the AV with a navigation instruction and optional notice instructions in natural language

LangAuto consists of three tracks:

• LangAuto track
: For each route, navigation instructions are given and updated to the agent based on the agent's current position.
: Tracks are divided into three sub-tracks with different route lengths, to better distinguish the performance.
(LangAuto - longer than 500 meters, LangAuto-Short - between 150 and 500 meters, and LangAutoTiny shorter than 150 meters)

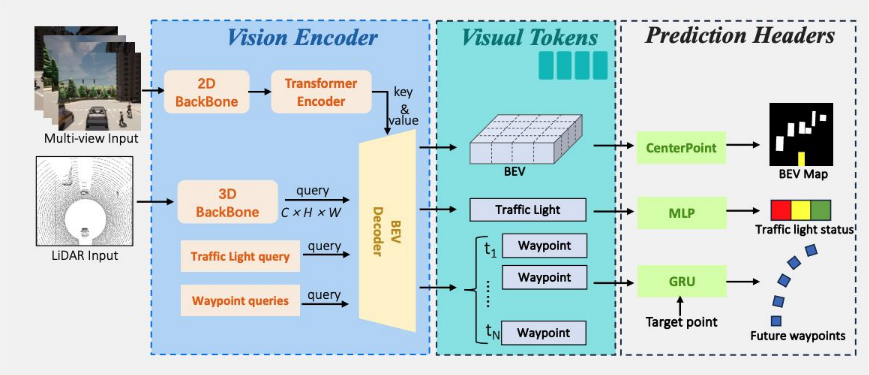
• LangAuto-Notice track
: Based on the LangAuto track, notice instructions added to the agent
: Give real-time notice in long-trail complex or adversarial scenarios(usually hard for the AV system to handle by itself)
Ideally, the agent that can comprehend and leverage instruction can achieve better performance.

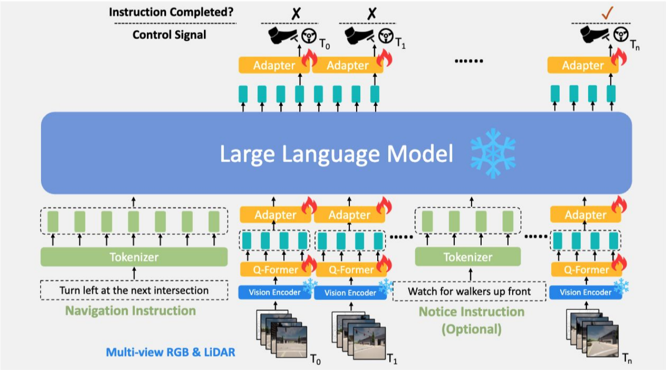
• LangAuto-Sequential track:
: Based on the LangAuto track, 10% of consecutive 2 to 3 instructions were merged into a single long instruction.
: This setting mimics the realistic scenarios where the multi-sentence instructions come from the passengers or navigation software.

: Misleading instructions will be randomly(~ 5%) and intermittently given to the driving agent, which lasts for a 1-2 seconds
: The driving agent is expected to reject these misleading instructions and execute safe actions, until the next correct instruction is spawned.

Experiments

Setting:





Vision encoder : ResNet-50 backbone is pre-trained on ImageNet, and PointPillars is trained from scratch.
: C, H, W, N are set as 256, 50, 50, 5 respectively.

Q-former token : Empirically found M = 4 tokens achieve decent performance.

Instruction-finetuning stage : Frames are sampled at intervals of 2
: 75% of clips have their notice instructions removed to prevent overfitting
: The remaining clips include at most one notice instruction.

Experiments

Quantitative Result

LLM Backbone	LangAuto			LangAuto-Short			LangAuto-Tiny		
	DS ↑	RC ↑	IS ↑	DS ↑	RC ↑	IS ↑	DS ↑	RC ↑	IS ↑
Random Init.	10.7±3.8	16.2±4.9	0.63±0.04	14.2±4.4	20.1±4.4	0.72±0.04	20.1±4.1	24.7±5.1	0.75±0.03
LLaMA [39]	31.3±1.5	37.1±1.6	0.82±0.01	42.8±7.2	49.1±8.5	**0.87±0.03**	52.2±5.3	57.8±8.0	**0.91±0.05**
LLaMA2 [40]	32.8±2.1	40.1±2.2	0.81±0.02	44.8±6.2	53.5±5.5	0.84±0.02	56.1±4.1	64.2±4.7	0.87±0.04
Vicuna [49]	33.5±1.9	39.3±1.9	0.83±0.02	45.3±4.9	54.3±3.9	0.83±0.03	55.5±3.9	63.1±4.2	0.88±0.04
Vicuna-v1.5 [49]	34.0±3.8	39.0±3.3	**0.85±0.06**	47.0±4.3	56.5±2.4	0.83±0.04	59.0±2.6	69.9±2.3	0.84±0.02
LLaVA-v1.5 [24]	**36.2±2.3**	**46.5±4.3**	0.81±0.03	**50.6±1.7**	**60.0±3.4**	0.84±0.04	**66.5±3.6**	**77.9±2.3**	0.85±0.02

: LLaMA, LLaMA2 - pre-trained with large public language datasets
: Vicuna, Vicuna1.5 - additionally fine-tuned with human conversation data
: LLaVA-1.5 - incorporate multi-modal data for training
⇒ LLaVA-1.5 >> Vicuna-v1.5 > LLaMA2 ≈ Vicuna > LLaMA >> Randomly initialized LLM models

Ablation study

Module design	DS ↑	RC ↑	IS ↑
Baseline (LLaVA-v1.5)	**36.2±2.3**	**46.5±4.3**	**0.81±0.03**
w/o Q-Former	31.7±3.5	41.2±4.4	0.79±0.02
w/o using BEV tokens	33.9±3.9	45.9±5.1	0.72±0.03
w/o visual pre-training	16.9±5.1	24.1±4.7	0.70±0.04

* RC(Route Completion) : percentage of the completed route length.
* IS(Infraction Score): Infractions triggered by the agent
* DS(Driving Score) : route completion ratio * the infraction score,

1) w/o Q-Former - Downsample the BEV features to 4×4 ⇒ The average driving score dropped from 36.2 to 31.7.
2) w/o using BEV tokens - Exclude the BEV tokens input ⇒ Results in a decreased infraction score 0.81 to 0.72
3) w/o visual pre-training - Train Vision encoder from scratch in the instruction-finetuning stage ⇒ The driving score drops to 16.9

Experiments

LangAuto vs LangAuto-notice

LLM Backbone	Benchmark Type	Infraction Score ↑	Vehicle Collisions ↓	Pedestrian Collisions ↓	Layout Collisions ↓	Red light Violations ↓	Offroad Infractions ↓	Blocked Infractions ↓
LLaVA-v1.5	LangAuto	0.81	0.33	0.03	0.50	0.92	0.36	0.22
	LangAuto-Notice	0.87	0.17	0.02	0.31	0.50	0.17	0.27
Vicuna-v1.5	LangAuto	0.85	0.30	0.03	0.43	1.18	0.24	0.19
	LangAuto-Notice	0.91	0.15	0.01	0.28	0.56	0.26	0.19

: Agents can leverage the notice instructions in real-time, resulting in a significant decrease in both collisions and traffic rule violations

LangAuto vs LangAuto-sequential

LLM Backbone	Benchmark Type	DS ↑	RC ↑	IS ↑
LLaVA-v1.5	LangAuto	36.2	46.5	0.81
	LangAuto-Sequential	34.0	43.7	0.81
Vicuna-v1.5	LangAuto	34.0	39.0	0.85
	LangAuto-Sequential	31.9	37.1	0.84

: Agents based on LLaVA and Vicuna both have a drop in the driving score and route completion ratio
: Sequential instructions additionally requires the agent to be temporally aware of which instruction has been completed or not.

감사합니다.